BOOTCAMP · DATA SCIENCE & FULL STACK · MÓDULO SQL

Proyecto de Creación de una Base de Datos Relacional

Explicación paso a paso: de unos CSV sin normalizar a una base de datos PostgreSQL alojada en la nube y lista para consultar.

Equipo E	Noa, Patricio, Nicolás y Paula (DS) · Mariela Velasquez Canache y Carlos Rodríguez Ruiz (FS)
Tipo de trabajo	Proyecto grupal
Datos de partida	5 ficheros CSV: `clase_1` … `clase_4` y `claustro`
Herramientas	draw.io · MySQL Workbench · PostgreSQL (Supabase)
Resultado	8 tablas · 420 filas · consultas de demostración

Índice

0 Antes de empezar: qué pide el proyecto y vocabulario básico

1 Los datos de partida

2 Paso 1 · Normalización y modelo Entidad-Relación

3 Paso 2 · Modelo lógico (relacional)

4 Paso 3 · Verificación de la normalización

5 Paso 4 · Creación de la base de datos (MySQL y PostgreSQL en Supabase)

6 Consultas de demostración

7 Escalabilidad: cómo crece la base de datos

8 Decisiones, limitaciones y mejoras futuras

A Anexo: ficheros entregados

> **Cómo leer este documento**
> Está pensado para que cualquier miembro del grupo pueda entender y defender el proyecto en la presentación, aunque no haya hecho la parte técnica. Cada paso explica **qué** se hace, **por qué** y **con qué resultado**, usando siempre nuestros datos reales.

0 · Antes de empezar

Qué nos pide el proyecto

Nos dan los datos de los alumnos de varias promociones del bootcamp (y de sus profesores) en ficheros CSV **sin normalizar**: todo mezclado en una tabla grande, con muchos datos repetidos. Tenemos que convertirlos en una **base de datos relacional** bien diseñada, que no repita información, que sea capaz de crecer (más campus, verticales, promociones, modalidades…) y que esté alojada en un servidor para poder consultarla desde otras aplicaciones.

Tarea del enunciado	Qué entregamos
1. Modelo Entidad-Relación (E/R)	Diagrama E/R hecho en draw.io
2. Modelo lógico	Esquema relacional: tablas, columnas, claves primarias y foráneas
3. Normalización	Análisis de la tabla original y comprobación de 1FN, 2FN y 3FN
4. Creación de la base de datos	Scripts SQL (MySQL y PostgreSQL) + base de datos en Supabase + consultas

> **Sobre el orden de los pasos**
>
> El enunciado pone la normalización en tercer lugar. Ese orden es el "clásico" de los libros, pensado para cuando *no* tienes datos y diseñas desde cero. Nosotros **sí** partimos de una tabla con datos, así que lo natural es **normalizar primero**: de la normalización salen solas las entidades del E/R. Por eso trabajamos así:
>
> 1. **Paso 1:** normalizamos la tabla original y de ahí sacamos el modelo E/R.
> 2. **Paso 2:** pasamos el E/R a tablas (modelo relacional).
> 3. **Paso 3:** volvemos a comprobar la normalización sobre las tablas finales (lo pide el enunciado y confirma que el diseño es correcto).
> 4. **Paso 4:** creamos la base de datos y cargamos los datos.

Vocabulario mínimo

Término	Qué significa (con un ejemplo nuestro)
Entidad	Una "cosa" de la que queremos guardar información. Ej.: *Alumno*, *Promoción*, *Proyecto*. Se convierte en una tabla.
Atributo	Una característica de una entidad. Ej.: el *email* de un alumno. Se convierte en una columna.
Relación	Cómo se conectan dos entidades. Ej.: un alumno *realiza* proyectos.
Cardinalidad	Cuántos elementos participan en una relación. **1:N** = uno con muchos; **N:M** = muchos con muchos (un alumno hace muchos proyectos y un proyecto lo hacen muchos alumnos).
Clave primaria (PK)	Columna (o columnas) que identifica cada fila de forma única. Ej.: `alumnos.id`.
Clave foránea (FK)	Columna que apunta a la PK de otra tabla, y así "enlaza" tablas. Ej.: `alumnosProyectos.id_alumnos` apunta a `alumnos.id`.
Redundancia	Guardar el mismo dato varias veces. Es lo que queremos eliminar.
Normalización	Proceso por pasos (1FN, 2FN, 3FN) para repartir los datos en tablas sin redundancia.

1 · Los datos de partida

Tenemos 5 ficheros CSV (separados por `;`) en la carpeta `data/`:

Fichero	Filas	Promoción	Campus	Proyectos que aparecen como columnas
`clase_1.csv`	15	Septiembre (18/09/2023)	Madrid	HLF, EDA, BBDD, ML, Deployment (Data Science)
`clase_2.csv`	10	Febrero (12/02/2024)	Madrid	HLF, EDA, BBDD, ML, Deployment (Data Science)
`clase_3.csv`	14	Septiembre (18/09/2023)	Madrid	WebDev, FrontEnd, Backend, React, FullSatck (Full Stack)
`clase_4.csv`	13	Febrero (12/02/2024)	Valencia	WebDev, FrontEnd, Backend, React, FullSatck (Full Stack)
`claustro.csv`	10	Profesores: nombre, rol (TA / LI), vertical, promoción, campus y modalidad		

Así es un CSV de alumnos (primeras filas de `clase_1.csv`)

Nombre	Email	Promoción	Fecha_comienzo	Campus	Proyecto_HLF	Proyecto_EDA	Proyecto_BBDD	Proyecto_ML	Proyecto_Deployment
Jafet Casals	Jafet_Casals@gmail.com	Septiembre	18/09/2023	Madrid	Apto	No Apto	Apto	Apto	Apto
Jorge Manzanares	Jorge_Manzanares@gmail.com	Septiembre	18/09/2023	Madrid	Apto	No Apto	Apto	Apto	Apto
Onofre Adadia	Onofre_Adadia@gmail.com	Septiembre	18/09/2023	Madrid	Apto	Apto	Apto	No Apto	Apto
… 12 filas más									

Así es el CSV del claustro (`claustro.csv`)

Nombre	Rol	Vertical	Promocion	Campus	Modalidad
Noa Yáñez	TA	DS	Septiembre	Madrid	Presencial
Saturnina Benitez	TA	DS	Septiembre	Madrid	Presencial
Mario Prats	LI	FS	Febrero	Valencia	Online
… 7 filas más · TA = Teacher Assistant, LI = Lead Instructor					

Cosas que nos fijamos al mirar los datos

- Una **promoción** queda definida por su **fecha de comienzo + campus + vertical**. Por ejemplo, "Septiembre 2023" existe en Madrid DS, Madrid FS y Valencia FS: son promociones distintas.
- La **vertical** (DS o FS) no viene escrita en los CSV de alumnos: se deduce de los proyectos (HLF, EDA… son de Data Science; WebDev, React… de Full Stack).
- El claustro menciona **dos promociones que no tienen alumnos** en los CSV: Valencia FS Septiembre y Madrid FS Febrero. Las creamos igualmente, porque tienen profesores.
- El nombre del proyecto `FullSatck` viene así (con la errata) en las cabeceras de los CSV y lo hemos mantenido tal cual.

2 · Paso 1: Normalización y modelo E/R

2.1 ¿Qué problemas tiene la tabla original?

Si metiéramos los CSV tal cual en una sola tabla, tendríamos esto (fíjate en lo que se repite):

Nombre	Email	Promoción	Fecha_comienzo	Campus	Proyecto_HLF	Proyecto_EDA	...
Jafet Casals	Jafet_Casals@...	Septiembre	18/09/2023	Madrid	Apto	No Apto	...
Jorge Manzanares	Jorge_Manzanares@...	Septiembre	18/09/2023	Madrid	Apto	No Apto	...
Onofre Adadia	Onofre_Adadia@...	Septiembre	18/09/2023	Madrid	Apto	Apto	...

Problema	Ejemplo con nuestros datos
Redundancia	"Septiembre · 18/09/2023 · Madrid" se repite en los 15 alumnos de `clase_1`.
Grupos repetidos	`Proyecto_HLF`, `Proyecto_EDA` ... son el mismo dato (una nota) en 5 columnas. Si mañana hay un proyecto nuevo, hay que *cambiar la estructura* de la tabla.
Anomalía de actualización	Si la promoción cambia su fecha de inicio, hay que cambiarla en 15 filas. Si se olvida una, los datos se contradicen.
Anomalía de inserción	No podemos dar de alta la promoción "Valencia FS Septiembre" porque todavía no tiene alumnos.
Anomalía de borrado	Si borramos al último alumno de una promoción, perdemos también los datos de esa promoción.

La normalización resuelve todo esto en tres pasos. Cada forma normal (FN) exige cumplir la anterior.

2.2 Primera Forma Normal (1FN): un solo valor por celda, sin columnas repetidas

Regla: cada celda guarda un único valor y no hay grupos de columnas que signifiquen lo mismo.

Qué hacemos: las 5 columnas `Proyecto_X` se convierten en **filas**: una fila por cada alumno y proyecto.

Antes (1 fila por alumno, 5 columnas de notas)

Email	Proyecto_HLF	Proyecto_EDA	...
Jafet_Casals@...	Apto	No Apto	...

Después (1 fila por alumno y proyecto)

Email	Proyecto	Evaluación
Jafet_Casals@...	HLF	Apto
Jafet_Casals@...	EDA	No Apto
Jafet_Casals@...	BBDD	Apto

Ahora, añadir un proyecto nuevo es añadir filas, no columnas. También convertimos las fechas de texto (`18/09/2023`) al formato de fecha estándar (`2023-09-18`) para guardarlas como `DATE`.

2.3 Segunda Forma Normal (2FN): cada dato depende de la clave entera

Regla: si la clave de una tabla está formada por varias columnas, ningún otro dato puede depender solo de una parte de ella.

En la tabla de 1FN, la clave es **(alumno, proyecto)**: hacen falta las dos cosas para saber una evaluación. Pero hay datos que dependen solo de una parte:

- **Nombre y email** dependen solo del **alumno** → van a la tabla `alumnos`.
- **El nombre del proyecto** depende solo del **proyecto** → va a la tabla `proyectos`.
- **La evaluación** sí depende de los dos (de qué alumno y de qué proyecto) → se queda en la tabla intermedia `alumnosProyectos`.

alumnos		
id	**nombre**	**email**
1	Jafet Casals	Jafet_Casals@...
2	Jorge Manzanares	Jorge_Manzanares@...

proyectos	
id	**nombre**
1	HLF
2	EDA

alumnosProyectos		
id_alumnos	**id_proyectos**	**evaluacion**
1	1	Apto
1	2	No Apto

Fíjate en que ya no escribimos "Jafet Casals" ni "HLF" muchas veces: usamos su **id**, que es un número corto.

2.4 Tercera Forma Normal (3FN): nada depende de algo que no sea la clave

Regla: un dato no puede depender de otro dato que no sea la clave (dependencia "transitiva").

En la tabla de alumnos todavía tendríamos promoción, fecha de comienzo y campus. Pero:

- La **fecha de comienzo**, el **campus** y la **vertical** no son datos del alumno: son datos de la **promoción**. Alumno → promoción → fecha. Esa cadena es la dependencia transitiva.
- Por eso sacamos todo eso a una tabla `promocion`, y relacionamos cada alumno con su promoción mediante la tabla `alumnosPromocion`.

- Igual con los profesores: su nombre, rol y modalidad van a `claustro`; a qué promoción dan clase va en `promocionClaustro`.
- Y qué proyectos tiene cada promoción va en `promocionProyectos`.

promocion				
id	año	fecha_comienzo	campus	vertical
1	2023	2023-09-18	Madrid	DS
2	2024	2024-02-12	Madrid	DS

alumnosPromocion	
id_promocion	id_alumnos
1	1
1	2

> **Resultado de la normalización**
> "Madrid · 18/09/2023 · DS" se guarda **una sola vez** (en la promoción 1). Si cambia la fecha, se cambia en un único sitio. Podemos crear promociones sin alumnos, y borrar un alumno no borra su promoción.

2.5 De la normalización salen las entidades

Cada tabla "principal" que ha salido de normalizar es una **entidad** del modelo E/R. Las tablas intermedias son **relaciones**:

Entidad	Atributos	De dónde sale
Alumnos	Id, Nombre, Email	Columnas Nombre y Email de `clase_X.csv`
Promoción	Id, Año promoción, Fecha de comienzo, Campus, Vertical, TipoModalidad	Columnas Promoción, Fecha_comienzo y Campus; la vertical se deduce de los proyectos
Proyectos	Id, Nombre proyecto	Cabeceras `Proyecto_X` de los CSV
Claustro	Id, Nombre, Rol, Modalidad	`claustro.csv`

Relación	Entre	Tipo	Se lee…
Pertenecen	Alumnos – Promoción	N:M	Un alumno pertenece a una o varias promociones; una promoción tiene muchos alumnos.
Realizan	Alumnos – Proyectos	N:M	Un alumno realiza muchos proyectos; un proyecto lo realizan muchos alumnos. Tiene el atributo **Evaluación**.
Tienen	Promoción – Proyectos	N:M	Una promoción tiene varios proyectos; un proyecto se da en varias promociones.
Tiene	Promoción – Claustro	N:M	Una promoción tiene varios profesores; un profesor puede estar en varias promociones.

> **¿Por qué Alumnos–Promoción es N:M y no 1:N?**
> Con los datos actuales cada alumno está en una sola promoción, pero en la realidad un alumno puede hacer más de un bootcamp (por ejemplo, Data Science y después Full Stack). Con N:M la base de datos ya está preparada para eso sin cambiar nada.

2.6 El diagrama Entidad-Relación

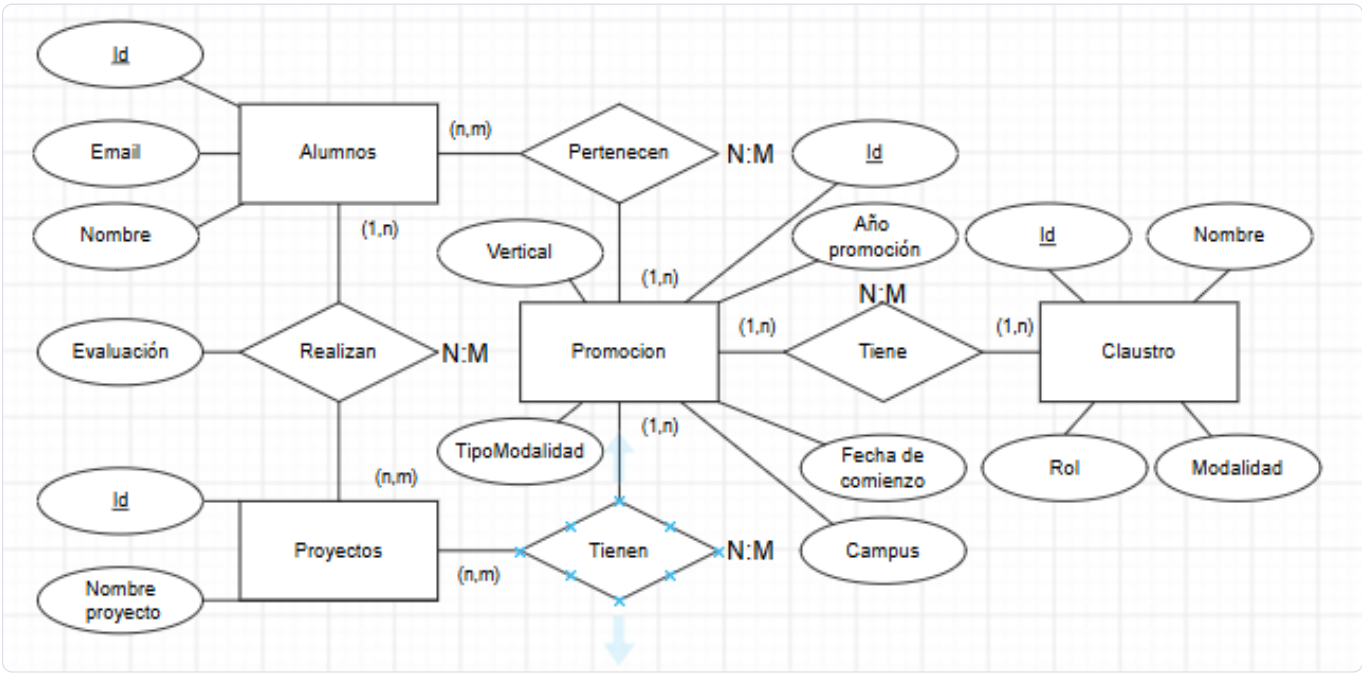


Figura 1 · Modelo Entidad-Relación del proyecto (draw.io)

Cómo se lee: rectángulo = entidad · elipse = atributo (subrayado = identificador, futura PK) · rombo = relación · elipse colgando de un rombo = atributo de la relación (**Evaluación** no es del alumno ni del proyecto, sino de "este alumno en este proyecto") · N:M = tipo de relación.

3 · Paso 2: Modelo lógico (relacional)

El modelo lógico traduce el diagrama E/R a **tablas**, con sus columnas, claves primarias (PK) y claves foráneas (FK). Como vamos a usar PostgreSQL, que es una base de datos relacional, el modelo lógico es el **modelo relacional**.

3.1 Reglas para pasar del E/R a tablas

En el E/R hay…	En el modelo relacional se convierte en…
Una **entidad**	Una tabla. Su identificador pasa a ser la **PK**.
Un **atributo**	Una columna de esa tabla.
Una relación **1:N**	No crea tabla: la PK del lado "1" se copia como **FK** en la tabla del lado "N".
Una relación **N:M**	Crea una **tabla intermedia** con las PK de las dos entidades. Juntas forman su PK, y cada una es FK a su tabla. Los atributos de la relación (como Evaluación) van en esta tabla.

3.2 Aplicación a nuestro modelo

Elemento del E/R	Tabla resultante
Entidad Alumnos	`alumnos`
Entidad Promoción	`promocion`
Entidad Proyectos	`proyectos`
Entidad Claustro	`claustro`
Relación N:M Pertenecen	`alumnosPromocion`
Relación N:M Realizan (+ Evaluación)	`alumnosProyectos` (con la columna `evaluacion`)
Relación N:M Tienen	`promocionProyectos`
Relación N:M Tiene	`promocionClaustro`

Como todas las relaciones son N:M, tenemos **4 tablas de entidades + 4 tablas intermedias = 8 tablas**.

3.3 Esquema relacional (en texto)

Formato: tabla (columnas). PK clave primaria · FK clave foránea.

Tabla	Columnas
`promocion`	**id** PK, año, fecha_comienzo, campus, vertical
`alumnos`	**id** PK, nombre, email
`proyectos`	**id** PK, nombre
`claustro`	**id** PK, nombre, rol, modalidad
`alumnosPromocion`	**id_promocion** PK FK → promocion, **id_alumnos** PK FK → alumnos
`alumnosProyectos`	**id_alumnos** PK FK → alumnos, **id_proyectos** PK FK → proyectos, evaluacion
`promocionProyectos`	**id_promocion** PK FK → promocion, **id_proyectos** PK FK → proyectos
`promocionClaustro`	**id_claustro** PK FK → claustro, **id_promocion** PK FK → promocion

3.4 Diagrama del modelo relacional

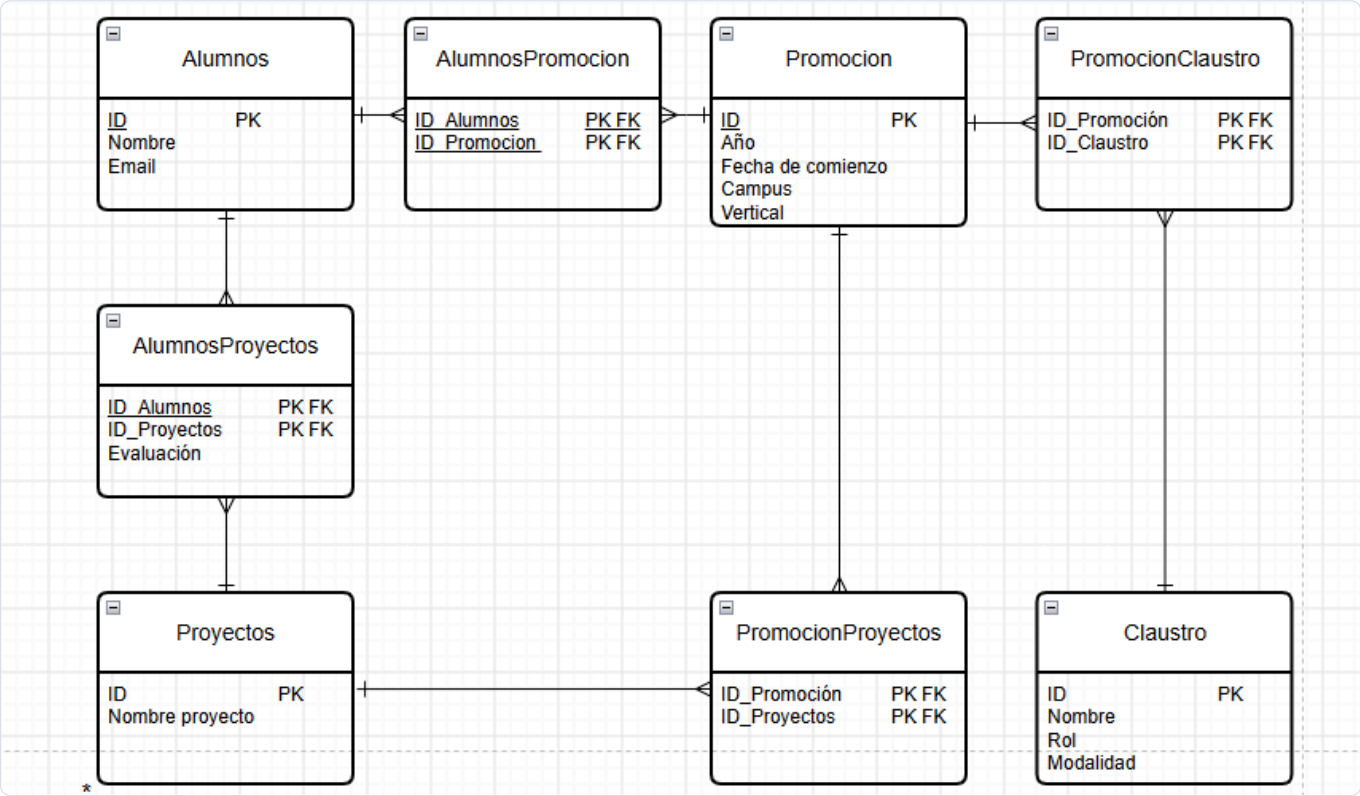


Figura 2 · Modelo lógico (relacional) del proyecto en draw.io: 8 tablas, con sus claves primarias (PK) y foráneas (FK)

> **Cómo leer una tabla intermedia**
> `alumnosProyectos` tiene 260 filas: una por cada pareja alumno–proyecto (52 alumnos × 5 proyectos). La fila `(1, 2, 'No Apto')` significa: "el alumno 1 (Jafet Casals), en el proyecto 2 (EDA), tiene No Apto". La PK compuesta `(id_alumnos, id_proyectos)` impide que un alumno tenga dos notas en el mismo proyecto.

Tipos de datos elegidos

Columna	Tipo	Motivo
`id` (todas)	INTEGER autoincremental	La base de datos genera el número sola al insertar.
`fecha_comienzo`	DATE	Permite ordenar y filtrar por fechas (no es un texto).
`nombre`, `campus`, `rol`, `evaluacion`	VARCHAR(n)	Texto corto con un tamaño máximo.
`email`	TEXT	Texto sin límite de longitud.
`modalidad`	ENUM('presencial','online') en MySQL · VARCHAR + CHECK en PostgreSQL	Solo admite esos dos valores.

4 · Paso 3: Verificación de la normalización

El enunciado pide analizar los datos y normalizarlos. Ya lo hicimos en el Paso 1 para diseñar el modelo; ahora **comprobamos sobre las tablas finales** que cumplen las tres formas normales. Si alguna fallara, tendríamos que volver a tocar el E/R.

Tabla	1FN valores únicos, sin grupos repetidos	2FN todo depende de la PK entera	3FN nada depende de algo que no sea la PK
`promocion`	✓	✓ PK simple	✓ (ver nota sobre "año")
`alumnos`	✓	✓ PK simple	✓ ya no guarda datos de la promoción
`proyectos`	✓	✓ PK simple	✓
`claustro`	✓	✓ PK simple	✓
`alumnosProyectos`	✓ una nota por fila	✓ la evaluación depende de alumno *y* proyecto	✓
`alumnosPromocion`	✓	✓ solo tiene la PK	✓
`promocionProyectos`	✓	✓ solo tiene la PK	✓
`promocionClaustro`	✓	✓ solo tiene la PK	✓

> **Conclusión**
> Las 8 tablas cumplen la 3FN, así que el diseño de los pasos 1 y 2 es correcto.

Cosas que revisamos durante la verificación

> **Error que encontramos y corregimos: la promoción guardada dos veces**
> En una versión anterior, la tabla `alumnos` tenía una columna `id_promocion` (relación 1:N) **y además** existía la tabla `alumnosPromocion`. Así, la promoción de cada alumno estaba guardada en dos sitios, que es justo la redundancia que queremos evitar. Lo corregimos dejando solo la relación N:M (`alumnosPromocion`) y quitando la columna de `alumnos`.

> **Decisión consciente: "año" en promoción**
> El año (2023) se puede sacar de `fecha_comienzo` (2023-09-18), así que estrictamente es un dato derivado. Lo mantenemos porque facilita las consultas por año, pero hay que tener en cuenta que, si se cambia la fecha, también hay que cambiar el año. Para que la base de datos sea más estricta se podría quitar y calcularlo con `EXTRACT(YEAR FROM fecha_comienzo)`.

> **Decisión consciente: modalidad solo para el claustro**
> Solo `claustro.csv` trae la modalidad (Presencial / Online), así que de momento la guardamos únicamente para el profesorado. En una promoción conviven profesores presenciales (los TA) y online (los LI), por eso **no** la pusimos en `promocion`.

Integridad de los datos

- **Claves primarias** en todas las tablas: no puede haber dos filas iguales.
- **Claves foráneas** en todas las tablas intermedias: no se puede poner una nota a un alumno o a un proyecto que no existan.
- **PK compuestas** en las tablas N:M: un alumno no puede estar dos veces en la misma promoción, ni tener dos notas en el mismo proyecto.
- **Tipos adecuados**: fechas como `DATE` y modalidad restringida a valores válidos.

5 · Paso 4: Creación de la base de datos

Primero la creamos en **MySQL Workbench** (en local) para probar el diseño. Después la pasamos a **PostgreSQL**, que es lo que pide el enunciado, y la alojamos en **Supabase**, un servicio gratuito de PostgreSQL en la nube (el enunciado sugiere Render; los dos sirven porque los dos son PostgreSQL).

5.1 Script de creación de tablas

El orden importa: primero se crean las tablas **sin FK** (promoción, alumnos, proyectos, claustro) y después las intermedias, porque una FK solo puede apuntar a una tabla que ya existe. Extracto del script MySQL:

```
CREATE TABLE promocion (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    año INTEGER,
    fecha_comienzo DATE,
    campus VARCHAR(20),
    vertical VARCHAR(5)
);
CREATE TABLE alumnos (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(20),
    email TEXT
);
-- Tabla intermedia N:M con atributo propio (evaluacion)
CREATE TABLE alumnosProyectos (
    id_alumnos INTEGER,
    id_proyectos INTEGER,
    evaluacion VARCHAR(20),
    PRIMARY KEY (id_alumnos, id_proyectos),
    FOREIGN KEY (id_alumnos) REFERENCES alumnos(id),
    FOREIGN KEY (id_proyectos) REFERENCES proyectos(id)
);
CREATE TABLE claustro (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(30),
    rol VARCHAR(20),
    modalidad ENUM("presencial","online")
);
-- ... y así el resto de tablas intermedias (alumnosPromocion, promocionProyectos, promocionClaustro)
```

5.2 Carga de los datos (INSERT)

Los `INSERT` se generan a partir de los 5 CSV, aplicando la normalización del Paso 1:

1. **Promociones:** cada combinación distinta de fecha + campus + vertical es una promoción (4 de los CSV de alumnos + 2 que solo aparecen en el claustro = **6**).
2. **Proyectos:** se sacan de las cabeceras `Proyecto_X` (**10**).
3. **Alumnos:** nombre y email de cada fila (**52**), y su promoción en `alumnosPromocion` (**52**).
4. **Evaluaciones:** cada celda Apto / No Apto pasa a ser una fila de `alumnosProyectos` (52 × 5 = **260**).
5. **Proyectos de cada promoción:** `promocionProyectos` (4 promociones × 5 = **20**).
6. **Claustro: 10** profesores con su modalidad, y su promoción en `promocionClaustro` (**10**).

```
INSERT INTO promocion (id, año, fecha_comienzo, campus, vertical) VALUES
    (1, 2023, '2023-09-18', 'Madrid', 'DS'),
    (2, 2024, '2024-02-12', 'Madrid', 'DS'),
    ...
INSERT INTO alumnosProyectos (id_alumnos, id_proyectos, evaluacion) VALUES
    (1, 1, 'Apto'),      -- Jafet Casals · HLF
    (1, 2, 'No Apto'),   -- Jafet Casals · EDA
    ...
```

5.3 De MySQL a PostgreSQL

MySQL y PostgreSQL usan casi el mismo SQL, pero hay diferencias que hacen fallar el script si no se adaptan:

En MySQL	En PostgreSQL	Por qué
`DROP/CREATE DATABASE` , `USE` , `SET NAMES`	Se quitan	Supabase ya nos da una base de datos (`postgres`) en UTF-8.
`AUTO_INCREMENT`	`GENERATED BY DEFAULT AS IDENTITY`	Es la forma estándar de PostgreSQL de numerar automáticamente.
`ENUM("presencial","online")`	`VARCHAR(20) CHECK (modalidad IN ('presencial','online'))`	PostgreSQL no admite ENUM dentro del CREATE TABLE, y las comillas dobles significan "nombre de columna", no texto.
Comillas invertidas `` `id` ``	Sin comillas	PostgreSQL no las entiende.
`SHOW TABLES`	Se quita	No existe en PostgreSQL (las tablas se ven en el Table Editor).
—	`DROP TABLE IF EXISTS … CASCADE` al principio	Para poder ejecutar el script varias veces sin errores.
—	`BEGIN; … COMMIT;`	Si algo falla, no se queda nada a medias.
—	`SELECT setval(...)` al final	Como insertamos los id a mano, hay que avisar al contador de que siga por el 53 (y no por el 1).

Un detalle: PostgreSQL guarda los nombres sin comillas en minúsculas, así que `alumnosProyectos` aparece como `alumnosproyectos` . Las consultas funcionan igual escribiéndolo de cualquiera de las dos formas.

5.4 Paso a paso en Supabase

1. 1 Entrar en **supabase.com**, crear un proyecto nuevo (nombre, contraseña de la base de datos y región).
2. 2 En el menú lateral, abrir **SQL Editor** → **New query**.
3. 3 Pegar el contenido completo de `ProyectoGrupal_PostgreSQL.sql` y pulsar **Run**.
4. 4 Comprobar el resultado de la consulta de recuento (abajo). Las tablas aparecen también en **Table Editor**.
5. 5 Para conectarse desde otras aplicaciones (Python, DBeaver, Power BI…), pulsar **Connect** y copiar la cadena de conexión. La contraseña no se comparte en el repositorio.

Resultado en Supabase: el *Schema Visualizer* (menú Database) dibuja automáticamente las 8 tablas y sus relaciones a partir de las claves foráneas. Es el mismo modelo relacional de la Figura 2.

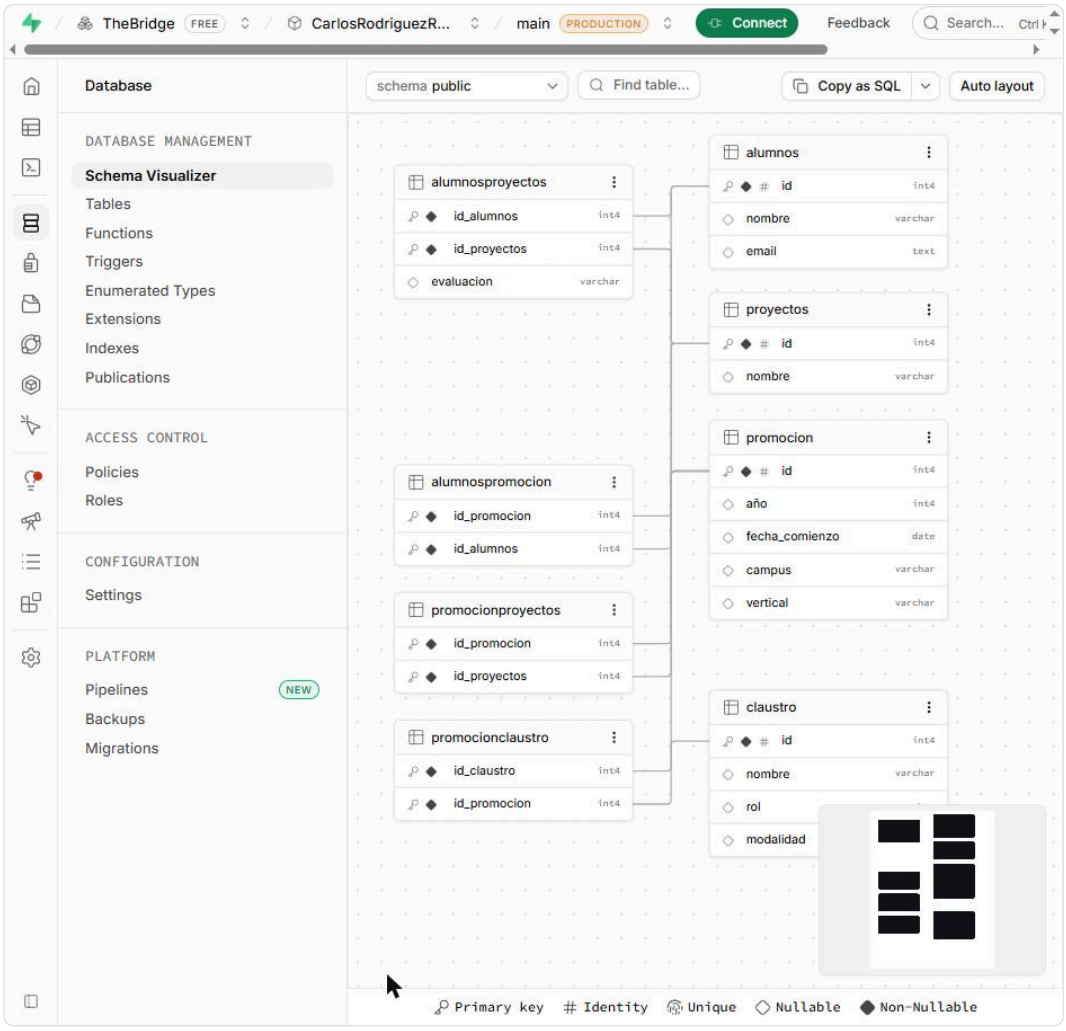


Figura 3 · Schema Visualizer de Supabase con las 8 tablas creadas

El mismo esquema, obtenido con la **ingeniería inversa de MySQL Workbench** (*Database → Reverse Engineer*), se ve mucho mejor que en Supabase: muestra el tipo de cada columna, marca las claves primarias y foráneas con iconos, y dibuja las relaciones con la notación "pata de gallo" (una raya = 1, tres patas = N), así que se ve de un vistazo qué lado de cada relación es el "1" y cuál el "N".

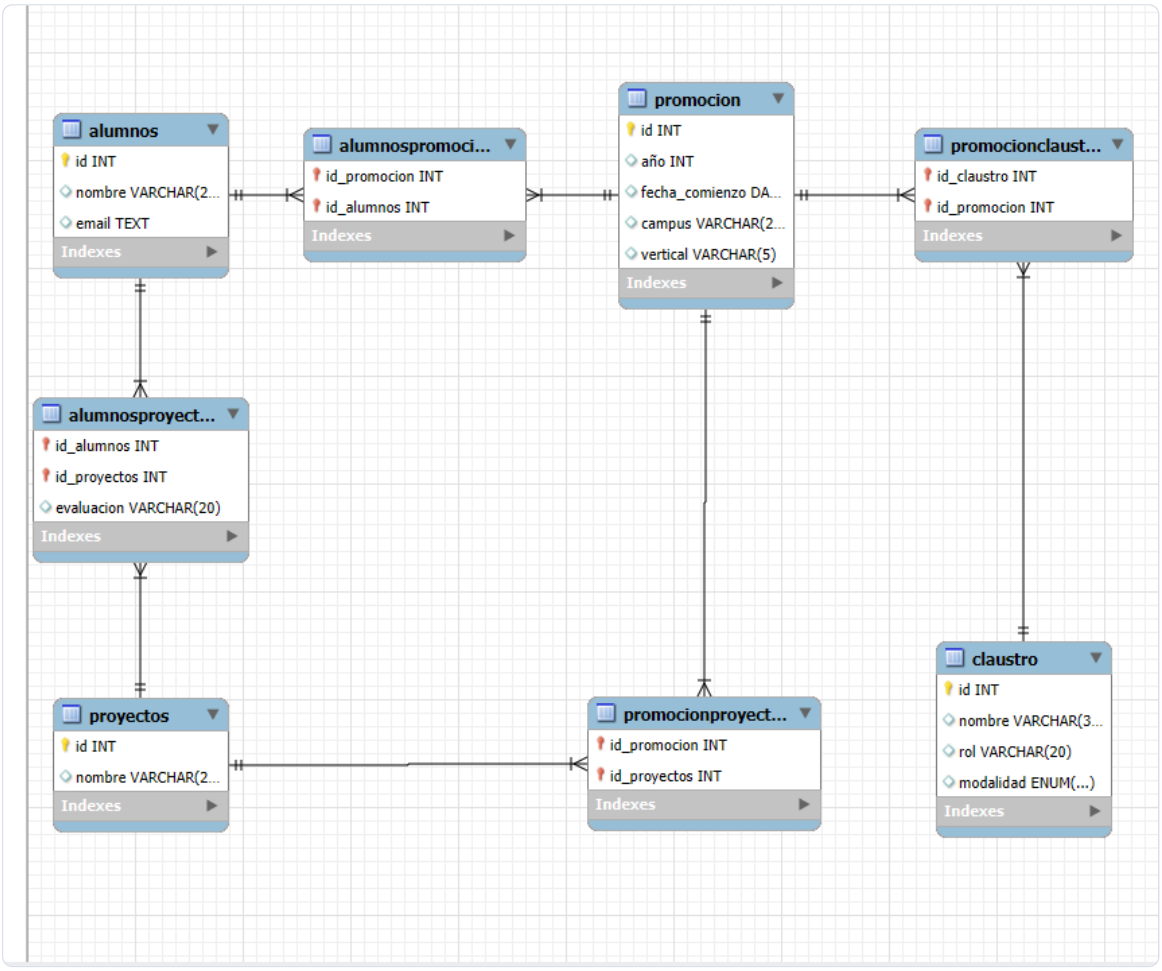


Figura 4 · Ingeniería inversa del esquema en MySQL Workbench

En el *Table Editor* se pueden ver y editar los datos de cada tabla. Por ejemplo, `claustro` con la modalidad de cada docente:

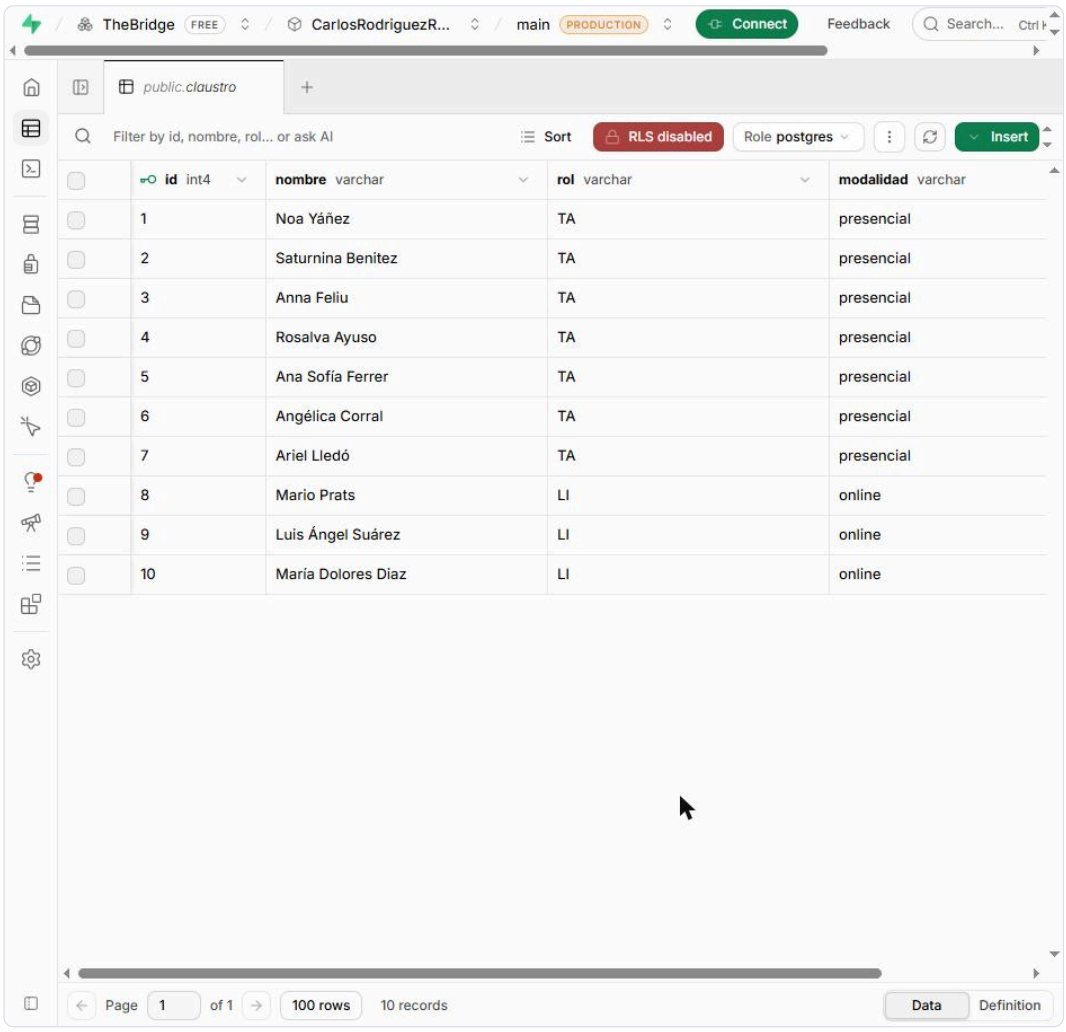


Figura 5 · Tabla claustro en el Table Editor de Supabase

5.5 Comprobación: ¿se han cargado todos los datos?

```sql
SELECT 'promocion' AS tabla, COUNT(*) AS filas FROM promocion
UNION ALL SELECT 'alumnos', COUNT(*) FROM alumnos
-- ... una línea por tabla
```

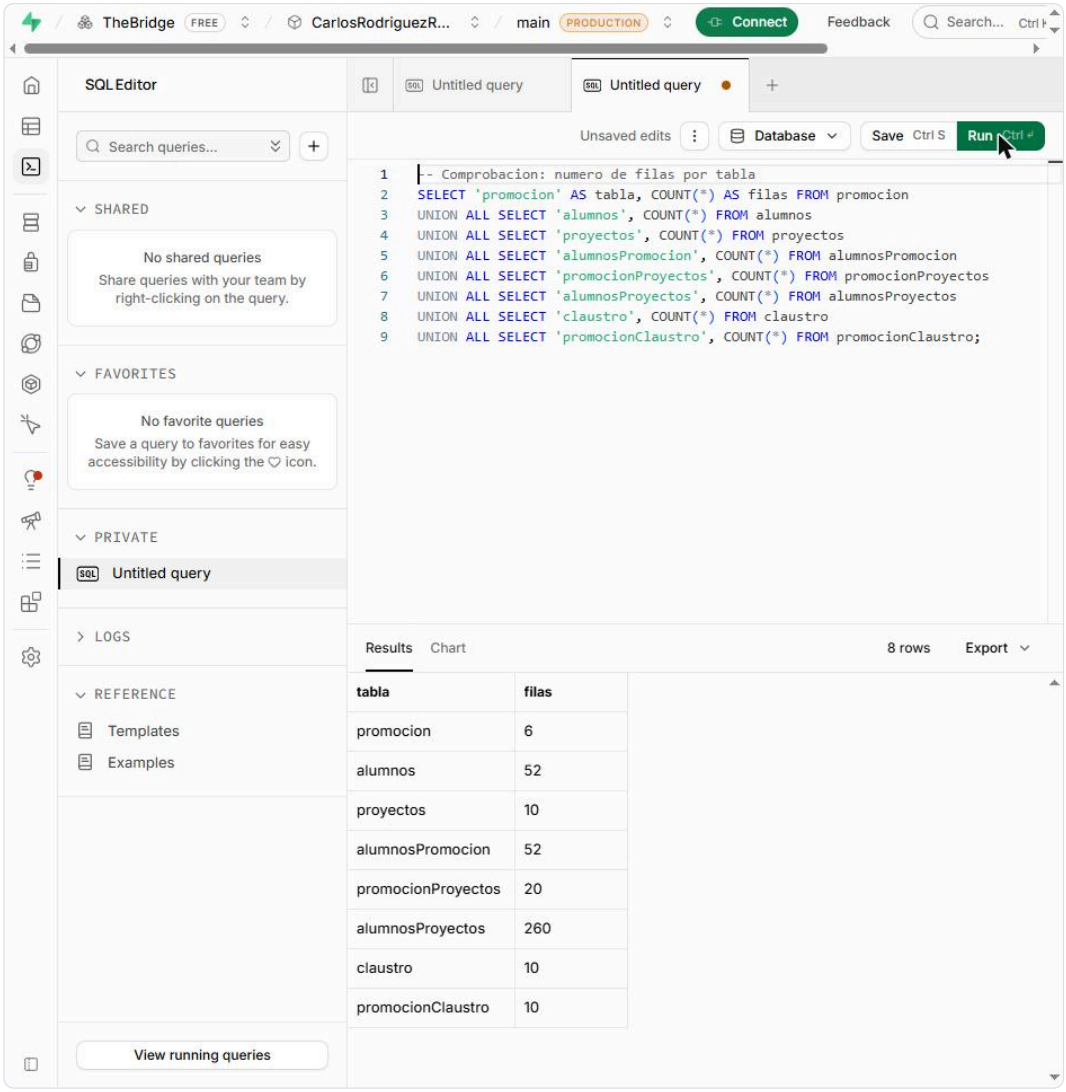


Figura 6 · Consulta de recuento ejecutada en el SQL Editor de Supabase

Total: 420 filas en 8 tablas
Coincide exactamente con lo que sale de los CSV.

6 · Consultas de demostración

Estas consultas son las que enseñaremos en la presentación para demostrar que la base de datos funciona. Todas usan `JOIN` para volver a juntar la información que la normalización repartió en varias tablas.

Consulta 1 · ¿Cuántos alumnos tiene cada promoción?

```sql
SELECT pr.id, pr.vertical, pr.campus, pr.fecha_comienzo,
       COUNT(apr.id_alumnos) AS num_alumnos
FROM promocion pr
    LEFT JOIN alumnosPromocion apr ON pr.id = apr.id_promocion
GROUP BY pr.id, pr.vertical, pr.campus, pr.fecha_comienzo
ORDER BY pr.id;
```

id	vertical	campus	fecha_comienzo	num_alumnos
1	DS	Madrid	2023-09-18	15
2	DS	Madrid	2024-02-12	10
3	FS	Madrid	2023-09-18	14
4	FS	Valencia	2024-02-12	13
5	FS	Valencia	2023-09-18	0
6	FS	Madrid	2024-02-12	0

6 filas

Usamos `LEFT JOIN` para que también salgan las promociones 5 y 6, que tienen profesores pero aún no tienen alumnos. Esto es justo lo que la tabla original no permitía (anomalía de inserción).

Consulta 2 · Porcentaje de aptos por proyecto

```sql
SELECT p.nombre AS proyecto,
       COUNT(*) AS evaluados,
       SUM(CASE WHEN ap.evaluacion = 'Apto' THEN 1 ELSE 0 END) AS aptos,
       ROUND(100.0 * SUM(CASE WHEN ap.evaluacion = 'Apto' THEN 1 ELSE 0 END) / COUNT(*), 1) AS porcentaje_aptos
FROM alumnosProyectos ap
    JOIN proyectos p ON ap.id_proyectos = p.id
GROUP BY p.id, p.nombre
ORDER BY porcentaje_aptos DESC;
```

EDA es el proyecto con más suspensos (56 % de aptos); ML y Deployment, los que más aprueban (88 %).

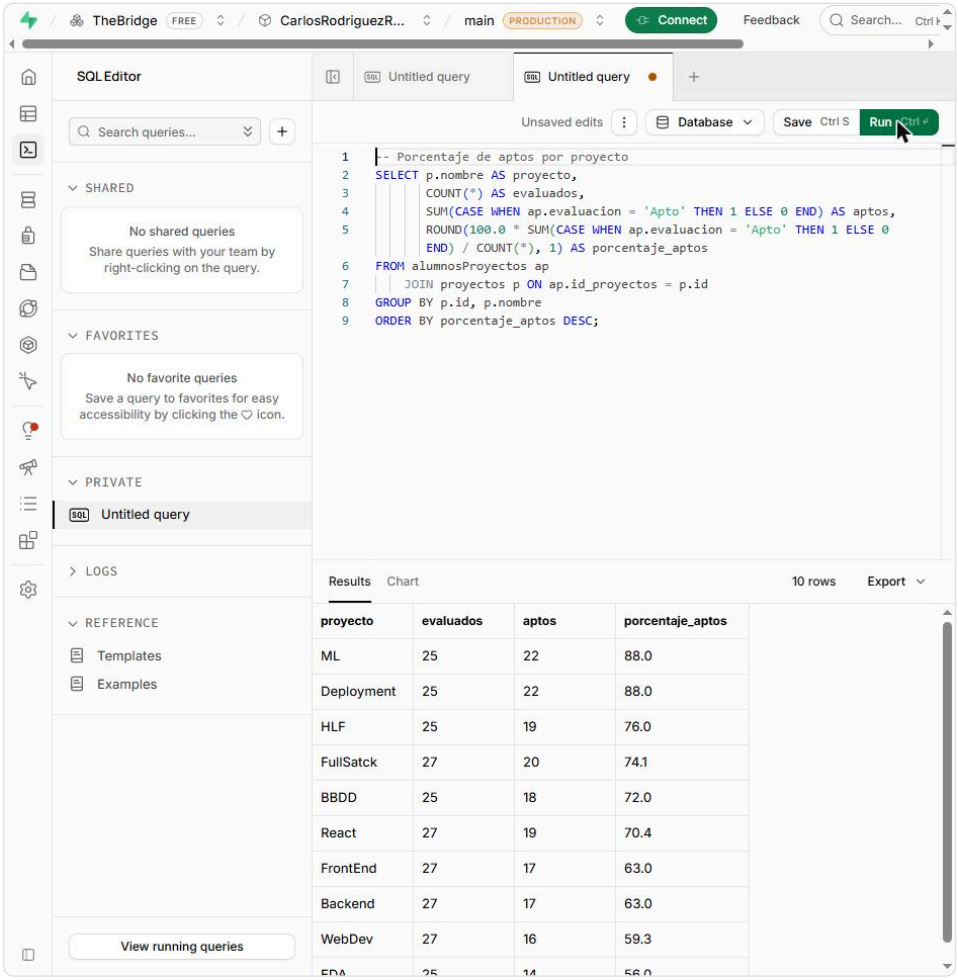


Figura 7 · Consulta 2 ejecutada en Supabase

Consulta 3 · Ficha de un alumno: promoción y notas

```sql
SELECT a.nombre, pr.vertical, pr.campus, p.nombre AS proyecto, ap.evaluacion
FROM alumnos a
    JOIN alumnosPromocion apr ON a.id = apr.id_alumnos
    JOIN promocion pr       ON apr.id_promocion = pr.id
    JOIN alumnosProyectos ap ON a.id = ap.id_alumnos
    JOIN proyectos p        ON ap.id_proyectos = p.id
WHERE a.nombre LIKE 'Jafet%'
ORDER BY p.id;
```

Con 4 `JOIN` reconstruimos la fila original del CSV, pero ahora a partir de datos sin redundancia.

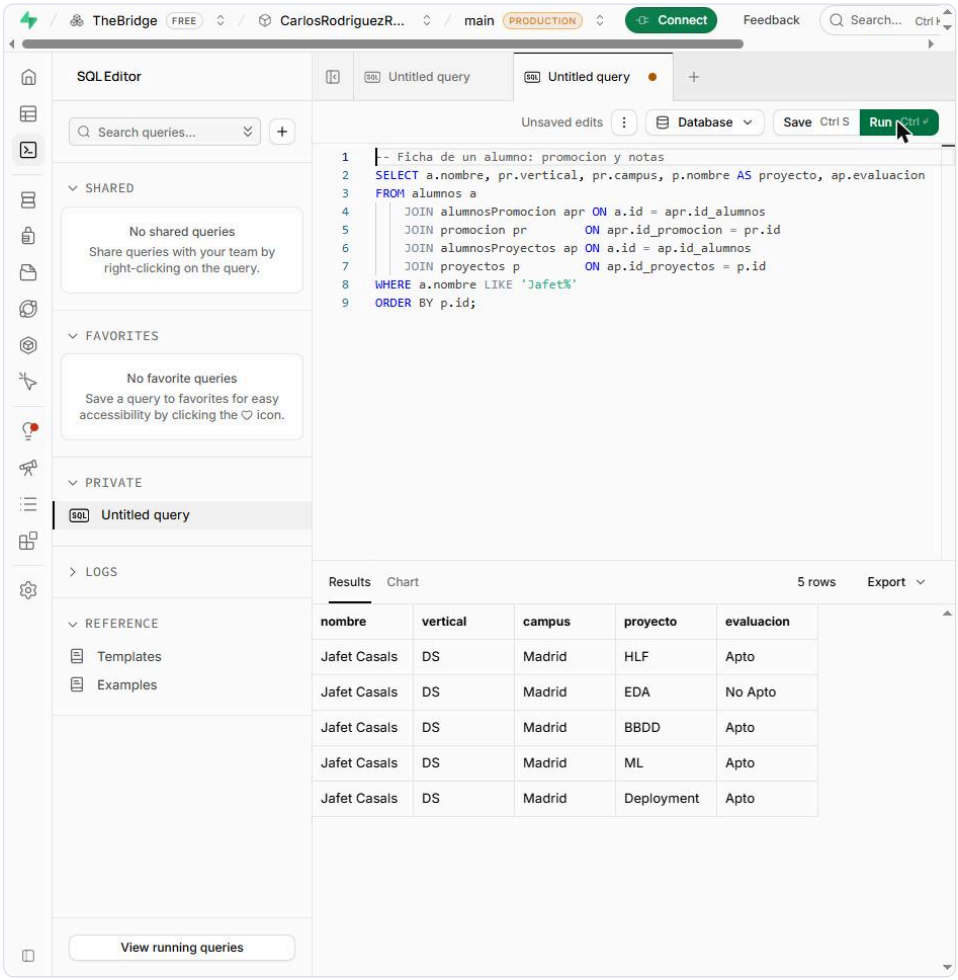


Figura 8 · Consulta 3 ejecutada en Supabase

Consulta 4 · Profesorado de cada promoción y su modalidad

```sql
SELECT pr.id AS promocion, pr.vertical, pr.campus, pr.fecha_comienzo,
       c.nombre AS docente, c.rol, c.modalidad
FROM promocion pr
    JOIN promocionClaustro pc ON pr.id = pc.id_promocion
    JOIN claustro c           ON pc.id_claustro = c.id
ORDER BY pr.id, c.rol DESC, c.nombre;
```

promocion	vertical	campus	fecha_comienzo	docente	rol	modalidad
1	DS	Madrid	2023-09-18	Ariel Lledó	TA	presencial
1	DS	Madrid	2023-09-18	Noa Yáñez	TA	presencial
1	DS	Madrid	2023-09-18	Saturnina Benitez	TA	presencial
1	DS	Madrid	2023-09-18	María Dolores Diaz	LI	online
3	FS	Madrid	2023-09-18	Anna Feliu	TA	presencial
3	FS	Madrid	2023-09-18	Luis Ángel Suárez	LI	online
4	FS	Valencia	2024-02-12	Ana Sofía Ferrer	TA	presencial
4	FS	Valencia	2024-02-12	Mario Prats	LI	online
5	FS	Valencia	2023-09-18	Rosalva Ayuso	TA	presencial
6	FS	Madrid	2024-02-12	Angélica Corral	TA	presencial

10 filas

Consulta 5 · Alumnos que han aprobado todos sus proyectos

```sql
SELECT a.nombre, pr.vertical, pr.campus
FROM alumnos a
    JOIN alumnosPromocion apr ON a.id = apr.id_alumnos
    JOIN promocion pr       ON apr.id_promocion = pr.id
    JOIN alumnosProyectos ap ON a.id = ap.id_alumnos
GROUP BY a.id, a.nombre, pr.vertical, pr.campus
HAVING SUM(CASE WHEN ap.evaluacion = 'No Apto' THEN 1 ELSE 0 END) = 0
ORDER BY a.nombre;
```

nombre	vertical	campus
Anita Heredia	DS	Madrid
Daniela Falcó	DS	Madrid
Eli Casas	DS	Madrid
Ernesto Uriarte	FS	Valencia
Haydée Figueroa	DS	Madrid
Herberto Figueras	FS	Madrid
Leoncio Tena	DS	Madrid
Loida Arellano	FS	Madrid
Ricarda Tovar	FS	Valencia
Rosalina Llanos	DS	Madrid
Teresa Laguna	FS	Valencia

11 filas

`HAVING` filtra después de agrupar: nos quedamos con los alumnos que tienen 0 "No Apto". Son 11 de 52.

Consulta 6 · Resumen por campus y vertical

```sql
SELECT pr.campus, pr.vertical,
       COUNT(DISTINCT apr.id_alumnos) AS alumnos,
       COUNT(DISTINCT pc.id_claustro) AS docentes
FROM promocion pr
    LEFT JOIN alumnosPromocion apr ON pr.id = apr.id_promocion
    LEFT JOIN promocionClaustro pc ON pr.id = pc.id_promocion
GROUP BY pr.campus, pr.vertical
ORDER BY pr.campus, pr.vertical;
```

campus	vertical	alumnos	docentes
Madrid	DS	25	4
Madrid	FS	14	3
Valencia	FS	13	3

3 filas

`COUNT(DISTINCT …)` evita contar dos veces a la misma persona al unir dos tablas a la vez.

7 · Escalabilidad: cómo crece la base de datos

El enunciado pide que la base de datos pueda crecer en campus, verticales, promociones, modalidades, etc. Gracias a la normalización, casi todo se hace **añadiendo filas, sin cambiar la estructura**:

Abrir un campus nuevo (Barcelona) con una promoción de Data Science

```
INSERT INTO promocion (año, fecha_comienzo, campus, vertical)
VALUES (2025, '2025-09-15', 'Barcelona', 'DS');   -- el id se genera solo
```

Crear una vertical nueva (Ciberseguridad) con sus proyectos

```
INSERT INTO proyectos (nombre) VALUES ('Redes'), ('Pentesting');
INSERT INTO promocion (año, fecha_comienzo, campus, vertical)
VALUES (2025, '2025-10-01', 'Madrid', 'CS');
-- y se enlazan en promocionProyectos
```

Un alumno que hace un segundo bootcamp

```
-- Jafet (id 1) se apunta también a la promoción Full Stack de Madrid (id 3)
INSERT INTO alumnosPromocion (id_promocion, id_alumnos) VALUES (3, 1);
```

Esto es posible porque Alumnos–Promoción es **N:M**. Con el diseño 1:N no se podría sin duplicar al alumno.

Quiero añadir...	¿Cambia la estructura?	Cómo
Un campus	No	Nueva fila en `promocion`
Una promoción	No	Nueva fila en `promocion`
Un proyecto	No	Nueva fila en `proyectos` + `promocionProyectos`
Una vertical	No	Nuevo valor en `promocion.vertical`
Un profesor	No	Nueva fila en `claustro` + `promocionClaustro`
Una modalidad nueva (p. ej. "híbrida")	Sí	Hay que modificar el ENUM / CHECK (ver mejoras)
Aulas	Sí	Nueva tabla `aula` relacionada con promoción (ver mejoras)

8 · Decisiones, limitaciones y mejoras futuras

Decisiones tomadas

- **Normalizar antes del E/R**, porque partimos de datos existentes (ver sección 0).
- **Todas las relaciones N:M**, incluida Alumnos–Promoción, para que un alumno pueda estar en varias promociones.
- **Crear las promociones 5 y 6** aunque no tengan alumnos, porque aparecen en el claustro.
- **Mantener los nombres tal como vienen en los CSV** (incluido `FullSatck`) para no alterar los datos de origen.
- **Supabase en lugar de Render**: los dos ofrecen PostgreSQL gratuito; el script funciona en ambos.

Mejoras que se podrían hacer

Mejora	Qué aporta
Tabla `modalidad` (id, tipo)	Añadir "Híbrida" o "Telemática" sería un simple INSERT, sin tocar la estructura. Se podría usar también para alumnos y promociones.
Tablas `campus` y `vertical`	Evitan errores de escritura ("Madrid" vs "madrid") y permiten guardar más datos de cada campus (dirección, ciudad...).
Tabla `aula`	El enunciado menciona las aulas como posible crecimiento.
Quitar `año` de promoción	Es derivable de `fecha_comienzo`; quitarlo elimina la última redundancia.
`UNIQUE` en `alumnos.email` y `NOT NULL` en columnas obligatorias	Más integridad: no puede haber dos alumnos con el mismo email ni alumnos sin nombre.
`CHECK (evaluacion IN ('Apto','No Apto'))`	Impide valores de evaluación inválidos.
Corregir `FullSatck` → `FullStack`	Limpieza de datos de origen.

Conclusión

Partimos de 5 CSV con datos repetidos y columnas que significaban lo mismo, y llegamos a una base de datos de **8 tablas en 3FN**, con claves primarias y foráneas que garantizan la integridad, **420 filas** cargadas, alojada en PostgreSQL (Supabase) y preparada para crecer en campus, promociones, proyectos y profesorado sin cambiar su estructura.

A · Anexo: ficheros entregados

Carpeta / fichero del repositorio	Contenido
`README.md`	Resumen del proyecto, equipo y cómo reproducirlo
`enunciado/Proyecto_BBDD.md`	Enunciado del proyecto
`data/`	Datos de partida: `clase_1.csv` … `clase_4.csv` y `claustro.csv`
`diagramas/`	Modelo E/R (Figura 1), modelo lógico (Figura 2), ingeniería inversa en Workbench y esquema en Supabase
`sql/mysql/ProyectoGrupalScriptCreacion.sql`	Creación de tablas y carga de datos para MySQL Workbench
`sql/postgresql/ProyectoGrupal_PostgreSQL.sql`	Script completo para Supabase / Render: tablas, datos y comprobación
`sql/consultas_demo.sql`	Las consultas de demostración de la sección 6
`python/carga_alumnos_pandas.py`	Carga alternativa de los CSV con pandas + SQLAlchemy (prueba en SQLite)
`docs/`	Esta memoria en PDF, la presentación y las capturas